



DPUSGEObject

Adding additional graphical objects to the virtual world.

1. Inputs:

<i>X,Y,Z</i>	Object's position in the simulation's world coordinate system [m]. Default: (0, 0, 0)
<i>rX,rY,rZ</i>	Object rotation around the x, y, and z axes [rad]. Default: (0, 0, 0)
<i>sX,sY,sZ</i>	Object scaling along the x, y, and z axes. Default: (1, 1, 1) (original object size)
<i>ColR, ColG, ColB, ColA</i>	Object color. Default: white (1, 1, 1, 1).
<i>Fog</i>	Influence of fog on the object. Default: 1 (normal influence of fog) Lower positive values improve the object's visibility in fog conditions.
<i>Show</i>	= 1 (default value): The object is visible. = 0: The object is (currently) invisible.

2. Inputs for each Parameter:

<i>Parami.X, Parami.Y, Parami.Z, Parami.W</i>	Coordinates X, Y, Z and W of the parameter value #i.
---	--

3. Static parameters:

<i>Object</i>	Resource name of the displayed SGE object (see "Functional principle").
<i>NMaxParams</i>	Number of shader parameters (0..10, default: 0)
<i>Visibility</i>	Visibility category for the object. It can be used to display the object on one of the views updated on a single graphics computer only. = 1 (default value): Always replace the graphical object's animation with SILAB's default animation to ensure that it can be transformed using the DPU inputs. = 0: Use the animation that has been specified in the object.
<i>UseDefaultAnimation</i>	

4. Functional principle:

Using DPUSGEObject, an additional graphical object (3D or 2D) can be inserted into the simulation. The object must be available in the SIG or SGE file format. An over-view on how such objects can be created is given in the manual "Reference Visualization SGE".



The object model must be located in a subfolder of

`SILAB\lib\graphics\default\models`. The parameter `Object` specifies the file name starting from this subfolder, where instead of the backslash character '\', dots '.' are used to separate folder names. The filename extension ".sig" or ".sge" is omitted.

5. Example:

`Object = "obstacles.bricks1"` corresponds to the object

`\SILAB\lib\graphics\default\models\obstacles\bricks1.sig`.

Additionally, the project path is searched for model files. Assuming that a project from `\SILAB\Projects\MyProject` has been loaded, the given model would also be found in

`\SILAB\Projects\MyProject\obstacles\bricks1.sig`.

Preferably, new models should be added to the project path, because it is not affected by SILAB upgrades.

Please make sure that the computer on which DPUSGEObject runs also executes DPUSGEWorld and DPUSGEView. The Index value of DPUSGEObject must be higher than the one of DPUSGEWorld, but less than the one of DPUSGEView.

The following DPU configuration section inserts the SGE object `\SILAB\lib\graphics\default\models\obstacles\gravel_pile.sig` into the simulation:

```
DPUSGEWorld SGEWorld
{
    Computer = {XYZ};
    Index = 50;
};

DPUSGEObject TestObject
{
    Computer = {XYZ};
    Index = 60;
    Object = "obstacles.gravel_pile";
    X = 10.0;
    Y = 0.0;
    Z = 0.0;
    Show = 1;
};

DPUSGEView SGEView
{
    Computer = {XYZ};
    Index = 98;
```



```
WindowW = 800;  
WindowH = 500;  
WindowX = 0;  
WindowY = 0;  
};
```

6. Shader parameters:

If the included object uses a special, user-defined vertex or pixel shader, parameters can be passed from DPU inputs directly to the shader.

To do so, the static DPU parameter NMaxParams must be used to create the re-quired amount of shader parameter inputs:

```
DPUSGEObject TestObject  
{  
    Computer = {XYZ};  
    Index = 60;  
    Object = "models.MyShaderObject";  
    X = 10.0;  
    Y = 0.0;  
    Z = 0.0;  
    Show = 1;  
  
    NMaxParams = 2;  
};
```

Each of these parameters is created as a vector of four components and thus has the “coordinates” X, Y, Z and W. However, these values are passed to the shader without any modification. Consequently, they can also be used to pass four unrelat-ed floating-point values.

Default values for shader parameters can be assigned in the DPU section. Of course, the shader parameters can also be connected within the SILAB system in order to modify the values dynamically.

```
DPUSGEObject TestObject  
{  
    Computer = {XYZ};  
    Index = 60;  
    Object = "models.MyShaderObject";  
    X = 10.0;  
    Y = 0.0;  
    Z = 0.0;  
    Show = 1;  
  
    NMaxParams = 2;  
  
    Param1.X = 10.0;
```



SILAB DRIVING SIMULATION

version Version 7.1 documentation

```
    Param1.Z = 231.29;  
    Param2.X = 13;  
};  
  
Connections = {  
    MyOscillator.Out <-> TestObject.Param1.Y  
};
```

Within the shader, these parameters must be declared as the variable *uniform vec4 Parami*; they can then be used to perform arbitrary calculations.